
PREDICT, THEN SIZE: DERIVING A DAILY POSITION RULE FROM A SIGNAL THAT ORDERS RETURNS BUT CANNOT SCALE THEM

This report records how a daily position rule was derived, one step at a time. It covers three things: how the features were built, how they and the model's output were diagnosed, and how the resulting prediction becomes a position.

Each step names the script that produced its numbers and the span they came from, gives the result, and says what the result forces the next step to measure. Reading only those hand-offs gives the whole argument. Code and cached output sit beside this file in the repository.

Setup

The data. Daily S&P 500 forward returns, the risk-free rate, and predictors named `<group><number>`, from the *Kaggle Hull Tactical — Market Prediction* series. The competition documents eight groups; this extract carries 93 raw columns across seven of them.

group	meaning	raw columns
E	economic and macro indicators	19
M	market and price-based indicators	18
P	positioning and valuation	13
V	volatility measures	13
S	sentiment signals	12
D	date and calendar flags	9
I	interest-rate and institutional series	9
MOM	momentum factors	—

The prefix MOM is matched before M, so a momentum series would never fall into the market group. Columns start at different dates, and the raw series runs 8990 rows against the feature table's 8918.

The split. `dev` (69 blocks, about 23 years, rows 1261–7056) fits models and measures mechanisms. `valid` (10 blocks, rows 7141–7980) chooses between configurations. `test` (11 blocks, rows 7981–8904) is read once, at the end. The development boundary is row 7113; the first holdout block starts at 7141. Part 1 touches neither holdout span.

The metric. The competition's adjusted Sharpe. A position $w \in [0, 2]$ earns $r_f(1 - w) + w f_r$, and the Sharpe of that series is divided by

$$\left(1 + \max(0, \sigma_s/\sigma_m - 1.2)\right) \times \left(1 + \text{gap}^2/100\right),$$

where σ_s/σ_m is realised strategy volatility over market volatility and `gap` is the shortfall of mean return against the market. Buy-and-hold is $w = 1$ and is also the market, so its adjusted Sharpe equals its plain Sharpe. For every configuration reported here both penalty factors come out at exactly 1.000, so the adjusted Sharpe below is the plain Sharpe.

Terms. `block_ic` is the mean per-block Spearman correlation between the signal and the next day's excess return, one block being the 84 rows a single fit predicts. `spread_ann` is the annualised top-minus-bottom-decile return, averaged over folds. `detectable` is twice the standard error of a paired contrast; a difference smaller than it cannot be separated from zero on the folds available. It is reported beside every contrast in place of a p -value.

Naming. The scripts that sweep the same parameter name it differently. This report uses `pipeline.py`'s names.

meaning	pipeline	mapping_form	detrend_sweep A/B	detrend_sweep C
rolling mean removed before ranking	detrend	mu_window	roll21	mean8, ewm5
the percentile's reference window	rank_window	rank_window	rank_win	rank_win
EWMA half-life on the position	smooth_half-life	—	—	smooth5

So $\text{mean8} \times \text{smooth5} \times \text{rank252}$ reads: subtract the mean of the last 8 predictions, take the percentile within the last 252 days, solve the scale, then apply an EWMA of half-life 5 days to the position.

1 The signal layer

1.1 Data processing

`Hull_Tactical_feature_engineering.py` (the whole table)

93 raw columns become 1132 features. What follows is what is built, on what window, and whether any of it can see the future.

Coverage. A column is kept when its data begins in the first 70% of the series, which admits late-starting predictors rather than truncating the panel to the shortest one. The per-group quantile rule is present and disabled.

Lagged returns. The three return series are shifted by one row before anything is built on them, so a signal at t uses returns through $t - 1$. They carry their own rolling block at ultra-short horizons (2, 3, 5, 10, 21), 74 columns.

Rolling statistics. Each raw column gets a rolling mean, z-score and range position at every window its group is assigned, plus one first difference. The windows are group-specific, because a volatility series and a macro series move on different clocks.

group	windows
V	5, 21, 63
S	5, 10, 21
M	5, 10, 21, 63
MOM	5, 21
E, P	63, 126, 252
I	21, 63
D	63, rolling mean only

D columns are 0/1 flags, so only the raw value and a rolling mean survive, which records how often the flag was on recently. The `_mean` columns of P, I and S are dropped once the gates are built, since the level of a slow series adds little beside its z-score and range position. The result is 289 z-scores, 267 range positions, 199 rolling means and 88 first differences.

Group PCA, refitted every row. Each group's z-scored columns are reduced to a first principal component. At row t the component is fitted on the window $[t - w, t)$ and applied to row t alone, so the fit never sees the row it transforms. Only columns with data somewhere in that window take part, which lets the active set grow as late-starting series come online, and the inputs are winsorised at the window's own 98th percentile before the fit.

group	method	window	warm-up	z-window / input
V	rolling	126	126	126 / <code>_z21</code>
M	rolling	126	126	126 / <code>_z21</code>
MOM	rolling	63	63	126 / <code>_z21</code>
E	rolling	252	252	252 / <code>_z63</code>
S	rolling	126	126	252 / <code>_z21</code>
I	expanding	—	504	252 / <code>_z21</code>
P	expanding	—	504	252 / <code>_z63</code>

I and P expand rather than roll because their series are slow. Each group produces `pc1`, its z-score and its positive part, 18 columns over the six groups with enough active inputs.

Regime gates. The PC1 factors are the regime variables:

$$\text{gate} = 1 + \text{clip}(\text{pc1_z}, 0, 3) \in [1, 4],$$

neutral at 1. Selected z-score columns are multiplied by it, so a feature counts for more when its own group’s regime is extreme and is left alone otherwise. The gated windows are short for the fast groups and long for the slow ones: V, S, M and MOM at 21 and 63, E and P at 126 and 252, I at 63 and 126. That adds 159 columns. Five hand-built interactions are also kept under an XINT_ prefix: I2 – I1, I7 – I1, P1/E1, V1/M1, S1 – S2.

Two assumptions worth naming. E and S columns are shifted by 10 rows before any window is taken, on the assumption that news in those blocks reaches prices with a delay. That is an assumption, not a measurement. And nothing is pruned: §1.4 removes each structural block in turn and every removal costs or ties, so all 1132 columns stay.

Now the causality question. Every dispersion statistic is computed on `s.rolling(w, min_periods=w)`, and the group factors come from `causal_pc1_onfly_winsor` above. The builder contains no `fit_transform` and no global mean or standard deviation, so a feature at row t uses only rows $\leq t$.

One dependency on the whole series survives. The coverage filter measures how late a column starts as a fraction of the table’s full 8918 rows, and drops it below 0.30. Normalising by the whole table rather than by development alone puts the threshold at row 6243 instead of 4979, and the stricter version would drop 71 more columns. The non-causal input is a scalar, “how long is the dataset”. No holdout value participates, and the latest first-valid row among surviving columns is 6039, well before the boundary at 7113.

The split, cut once

`split_data.py` (after §1.1, before everything else)

The feature table is cut into three files, and every later script reads a span through `paths.load_split(span)`. The geometry is derived from `wfo.make_folds` under the frozen constants rather than written down, so it cannot drift from what the models do.

	rows kept	scored	folds	history
train	504–7112	1261–7056	69	757
valid	6384–7980	7141–7980	10	757
test	7224–8917	7981–8904	11	757

A span’s models train on rows that precede it, so each file carries its scored rows plus the 757 rows of history those fits need: the training window, the purge and the embargo. The files therefore overlap, and any one of them runs the full walk-forward for its span alone. Each file is an exact slice, so `feature_cols` still returns 1132 and `manifest.json` records where each slice begins.

Three entry points cover the later uses. `load_span` returns a slice and its folds, indexed into it. `load_through` returns every row from the first training row through the end of a span, which one fit on all of development needs because it trains on 6636 rows while `valid`’s own slice carries 757. `remap_rows` and `slice_covering` place a script’s own fold grid onto a slice and refuse, loudly, when the slice lacks the history that grid needs.

The split runs after the builder because the builder drops the leading warm-up rows, `date_id` 0 to 71, where the longest rolling windows have no history. The raw series has 8990 rows and the feature table 8918, and the fold grid is positional on the feature table, so a split computed on the raw index lands 58 rows away from where the models cut.

`test` is now unreachable unless a script names it, and nothing before the final scoring does. That closes two failure modes: a holdout row reaching a fit through a trailing statistic, and a fold grid quietly losing folds because a slice was too short to hold them.

1.2 Check that the group factors are still factors

`pc1_check.py` (train split)

`MIN_COVERAGE = 0.30` lets late-starting columns back in, so each group’s PCA active set now grows several times. Counting sign flips and jumps in PC1 as columns come online puts both at about 4–5% of rows. A rolling z-score absorbs a scale change but not a sign change, so that rate is the thing to watch.

Small enough to keep the factors, large enough to record: `V_pc1_z126` and `E_pc1_z252` are downstream of it.

► **Leads to.** The features are usable. Before asking which of them matter, the model that answers has to be large enough to reach them.

1.3 Fix the model's capacity first

`capacity_window_grid.py` (*dev, 69 folds, causal trailing percentile*)

Training window against leaf count, as a 3×5 grid rather than two ladders, because the axes interact: the capacity gain from 5 to 62 leaves measured $+0.442$ at a 756-row window and $+0.124$ at an expanding one. Scored by annualised decile spread, with paired contrasts on shared folds.

window \ target	5	20	62	117	176
252	0.183	0.519	0.670	—	—
504	0.446	0.672	0.672	0.626	—
756	0.364	0.699	0.704	0.854	0.928

contrast	diff	<i>t</i>	detectable	separable
@756: T62 – T5	+0.511	2.11	0.507	yes
@756: T117 – T5	+0.669	2.52	0.555	yes
@756: T176 – T5	+0.737	2.84	0.545	yes
@T5: W252 – W756	−0.171	−0.76	0.464	no
@T20: W252 – W756	−0.247	−1.11	0.458	no
@T62: W252 – W756	−0.180	−0.79	0.470	no
@T117: W504 – W756	−0.039	−0.17	0.491	no
W756T176 – W756T62	+0.195	1.85	0.215	no

Capacity separates and the window does not. Above 62 leaves the gains stop separating from each other, so 62, 117 and 176 form one band rather than a ranking. Frozen at window 756, target 117: the middle of that band, at the longest window. `embargo_capacity.py` later re-swept $\{5, 20, 62, 117\}$ at embargo 0 and put 117 top on dev again, `block_ic` 0.1011 against 0.0943 at 62.

► **Leads to.** The model is now large enough that an ablation reports on the features rather than on the model. §1.4 shows what the reversed order produces.

1.4 Ablate the feature blocks

`ablate_blocks.py` (*dev, 69 folds, window 756, target 117, causal trailing percentile*)

Remove one structural block at a time, at a capacity the model can use. On a 756-row window:

	splits/tree	distinct columns/tree	columns ever used
4 leaves	3	2.9	267 of 1132 (24%)
92 leaves	91	65.7	666 of 1132 (59%)

A model that touches a quarter of the columns cannot report on the other three quarters. This runs at 117.

removed	columns	spread_ann	vs baseline	<i>t</i>	detectable	at 4 leaves
baseline	0	0.854	—	—	—	—
D group	18	0.707	−0.187	−2.81	0.136	−0.067
long_mean	97	0.754	−0.095	−1.32	0.147	+0.013
all_pos	261	0.691	−0.172	−1.78	0.197	+0.024
long_z	64	0.864	+0.073	+0.98	0.152	+0.037
gates	159	0.699	−0.109	−1.32	0.169	−0.051

Removing any block costs or ties, and nothing gains. Only the D group separates, and it is a loss, so all 1132 columns stay. The last column is the same measurement at four leaves, where every effect sits inside ± 0.067 : a uniform null that says nothing about the features and everything about the model producing it.

► **Leads to.** Capacity and features are fixed. What remains of the model is the hyperparameters underneath them.

1.5 The remaining tree hyperparameters

tree_sensitivity.py (dev, 69 folds, one axis at a time, within-block ranking)

axis	default	best alternative	difference	detectable
learning_rate	0.05	0.01	−0.060	0.198
colsample_bytree	0.7	0.9	−0.065	0.242
reg_lambda	5.0	1.0	−0.051	0.210
subsample	0.7	0.85	−0.176	0.244
n_estimators	250	500	+0.010	0.065

Every deviation is negative or inside its own resolution, so the defaults stand. The extremes are genuine losses: learning_rate 0.20 at -0.528 ($t = -4.22$), reg_lambda 100 at -0.651 , colsample 1.0 at -0.268 ($t = -2.70$).

► **Leads to.** The model is fully specified. What decides the rest of the project is which part of its output is usable: the number, or only the ordering.

1.6 What kind of signal this is

ridge_signal.py (both models), signal_diagnostics.py (LightGBM) (dev, 69 folds)

Three measurements. The level: R^2_{os} against zero and against the trailing mean, then the Mincer–Zarnowitz slope with Newey–West errors, then that slope re-estimated from earlier rows only and applied forward, which tests whether the overstatement was knowable at the time. The order: per-block Spearman. The increment: inside each 84-row block, r is the normal score of the within-block rank and $m = (s - \text{block mean})/\text{block sd}$, so r carries the order alone and m carries order and magnitude together.

level	sd(pred)	sd(truth)	R^2 vs zero	vs trailing mean
ridge	0.0062	0.0110	−0.261	−0.260
tree	0.0044	0.0110	−0.119	−0.117

	MZ slope b	NW t	$1/b$	causal b	R^2 after causal rescale
ridge	0.0872	3.03	11.5×	0.0899	+0.0015
tree	0.1393	2.54	7.2×	0.1431	+0.0015

order	block_ic	t	blocks positive	pooled IC
ridge	0.1060	10.93	91.3%	0.0448
tree	0.1011	8.11	87.0%	0.0531

increment	model	R^2	t on r	t on m
ridge	r only	0.0108	8.36	
	m only	0.0109		8.28
	both	0.0110	0.46	0.74
tree	r only	0.0094	7.57	
	m only	0.0117		8.10
	both	0.0136	−2.41	+3.52

A linear signal exists at $t = 3.03$, and the prediction overstates it by $11.5\times$. The overstatement was knowable at the time, which is why R^2 moves from -0.261 to $+0.0015$ once the slope is estimated causally and applied forward. The order is strong. For the Ridge, adding the magnitude to the order buys $+0.0002$ of R^2 and neither coefficient survives, so the magnitude can be thrown away for free. Keep the rank and map it through a percentile. Every rule that converts a prediction into return units before sizing is out.

For the tree, m stays significant and joint R^2 rises from 0.0094 to 0.0136, so a rank-based mapping handicaps it specifically. Part 2's search therefore owes the tree a magnitude encoding; §2.4 settles that debt before dropping it.

► **Leads to.** The signal layer is closed and produces one number per day, an ordering. Part 2 asks what position that ordering becomes, starting with the shape of the curve.

2 The mapping layer

2.1 Every shape of the rank-to-position curve works

mapping_form.py --curves (dev, 69 folds, $\bar{w} = 1.0$, scale solved causally to a volatility ratio of 1.2)

Part 1 leaves one number per day, where today sits among the last 252. Seven curves from the literature turn that into a position: AFML sigmoid sizing, Gu–Kelly–Xiu extreme-decile spread, dead bands and the rest. Each gets its own causally solved scale, so none can win by taking more risk. The shapes differ substantially:

p	linear	normal score	decile	extreme decile	dead band
0.02	0.31	0.21	0.34	0.31	0.02
0.25	0.64	0.74	0.63	1.00	1.00
0.50	1.00	1.00	1.07	1.00	1.00
0.75	1.36	1.26	1.37	1.00	1.00
0.98	1.69	1.79	1.66	1.69	1.98

curve	pooled adj	vs market	t	vs linear	detectable	turnover	days with a view
normal score	0.6103	+0.2817	3.22	+0.0019	0.045	0.257	98.6%
linear	0.6015	+0.2799	3.25	—	—	0.274	98.6%
decile 10	0.5866	+0.2813	3.36	+0.0014	0.021	0.271	100%
multi-horizon	0.5862	+0.2942	3.57	+0.0143	0.050	0.288	99.0%
extreme decile	0.5807	+0.2820	2.79	+0.0021	0.143	0.190	21.5%
dead band	0.5777	+0.2798	3.24	−0.0000	0.132	0.159	21.5%
isotonic	0.4788	+0.2032	4.60	−0.0766	0.128	0.118	95.7%

All seven beat buy-and-hold, by +0.28 per fold at t between 2.8 and 4.6, and pooled 0.60 against the market’s 0.406. None separates from any other: the tightest null is decile 10 at +0.0014 against a detectable of 0.021. Turnover varies by 1.8× and time in the market by 4.6× with no change in score. Once the risk budget is fixed, any monotone transform of the rank extracts the same information and the shape only moves where the risk is taken. Take the linear curve and stop tuning shape.

This also sets the baseline the rest of Part 2 has to explain. Carried forward unchanged, this mapping scores −0.390 on valid, so the shape is not what fails out of sample.

One residual. The normaliser divides by a full-sample standard deviation, but the causal solver cancels it exactly: scaling z by c makes the solver return τ/c , verified at 4.4×10^{-16} beyond row 120. The residual covers 120 of 5796 rows and is identical across all seven candidates. The isotonic fit is refitted every 84 rows on rows strictly before the block, and no fold reaches past row 7066.

► **Leads to.** The curve is fixed and its amplitude is still free. The metric has an opinion about amplitude, so that comes next.

2.2 How much of the signal can be deployed

mapping_form.py --params (dev, 69 folds; LightGBM evidence, corroborated on the Ridge in \$A)

The position is $w = 1 + \tau z$, with τ solved so w runs at a chosen ratio of market volatility. The metric taxes ratios above 1.2 linearly, so the question is whether to stop at the kink. Reading the unpenalised Sharpe answers it, because the tax is divided out and what is left measures the signal.

budget	adjusted	unpenalised Sharpe	vol penalty	mean τ
1.0	0.410	0.410	1.000	0.009
1.1	0.550	0.550	1.000	0.291
1.2	0.602	0.602	1.000	0.464
1.3	0.572	0.617	1.080	0.609
1.4	0.541	0.617	1.140	0.740
1.5	0.524	0.616	1.176	0.862
1.7	0.509	0.619	1.216	1.090
2.0	0.495	0.617	1.246	1.409

how τ is obtained	pooled adjusted	realised ratio	vol penalty
one solve per fold, held 84 days	0.6281	1.234	1.034
re-solved every day	0.6773	1.199	1.000

The unpenalised Sharpe climbs +0.19 between ratios 1.0 and 1.2, then sits at 0.617 across a doubling of the swing while the tax keeps growing. Past about 1.3 a wider swing buys variance and no return. So $\bar{w} = 1.0$, budget 1.2, τ re-solved daily on a trailing 756 rows. Crossing the kink is out, and so is picking τ by hand.

The daily re-solve is worth +0.049 pooled, the same order as the budget itself, while its per-fold paired difference is -0.0096 against a detectable of 0.115. Only the pooled number can see it, because the metric's penalties are computed on whatever window they are handed. The solver's own constants move the result by less than ± 0.005 : `tau_min_hist` 60/120/252 $\rightarrow$ 0.6054/0.6015/0.6011, `tau_fallback` 0.0/0.3/0.6/1.0 $\rightarrow$ 0.6008/0.6015/0.6020/0.6037, solver window 252/504/756 $\rightarrow$ 0.580/0.600/0.602. As a check, budget 1.0 scores 0.410 with mean $\tau = 0.009$, reproducing the market's 0.406.

The saturation is a property of the signal and carries to any span. The value 1.2 is this metric's kink and carries nowhere.

► **Leads to.** Curve and amplitude are fixed. One degree of freedom is left, what the percentile is measured against, and the next four steps are spent on it.

2.3 How much is there to gain from the reference set

`detrend_sweep.py --grid (Ridge, dev, 69 folds)`

The shipped rule ranks today against the trailing 252 days after removing a 63-day rolling mean. The alternative reference is the 84 rows this fit produced, since each model predicts exactly one block and block and fit are the same thing. Re-ranking the same predictions against each reference prices the difference, and splitting the within-block rank into the block's mean alone and then the full rank gives two look-ahead ceilings.

reference	block_ic
trailing 252, after a 63-day mean (shipped)	0.0756
trailing 252, no mean removed	0.0950
the block's actual mean removed (look-ahead)	0.1002
the full within-block rank (look-ahead)	0.1060

Ranking against a trailing window with nothing removed lands within 0.0053 of knowing the block's true centre, and within 0.0110 of the full within-block rank. The Ridge therefore has almost no level problem. The shipped rule gives away 0.0247 against perfect centring, and 0.0194 of that, four fifths, is the 63-day mean damaging the signal rather than a level the reference set fails to remove. For this model, 63 is the wrong length.

That also closes the whole line of work that estimates a fit's output level and subtracts it. A perfect level estimate is worth 0.0053 here, and `level_refit.py --model ridge` prices the best available one directly: it correlates at +0.342, which is 2.9 standard errors and therefore genuinely not flat, but its dispersion is a fifth of the truth's, 0.0009 against 0.0042. It captures roughly 12% of the level variance and buys about +0.0015 `block_ic`.

► **Leads to.** Two questions fall out. Those same level estimators behave very differently on the two candidate models, which decides which model ships (§2.4). And 63 being wrong raises what is right, which needs a criterion first (§2.5).

2.4 Why the model is a Ridge

`one_model.py, level_refit.py (both --model {tree,ridge}), detrend_sweep.py (dev + valid)`

Both models order returns about equally on dev: `block_ic` 0.1060 for the Ridge against 0.1011 for the tree (§1.6). Running the same diagnostics on both, from one script each with a `--model` switch, holds everything but the fit constant: the same splits, the same fold grid at embargo 0, both fitted fresh.

Hold the mapping fixed and change only how the signal is produced, over identical valid rows. `p_window` is the within-block rank, which is look-ahead and immune to the block's level by construction, so it bounds what any causal rule could reach.

signal produced by	tree, causal	tree, ceiling	Ridge, causal	Ridge, ceiling
refit every 84 rows, 756-row window	0.0062	0.0468	0.0984	0.1026
1 fit, 756-row window	0.0399	0.0780	0.0843	0.0861
1 fit, ~6600-row window	0.0668	0.0896	0.0425	0.0835

The two ladders run in opposite directions. The tree’s causal ordering improves tenfold as refits are removed; the Ridge’s is best with the most refits and decays as they are taken away.

Under the shipped protocol the tree sits at 0.0062 against a ceiling of 0.0468, a gap of 0.041, while the Ridge sits at 0.0984 against 0.1026, a gap of 0.004. The tree’s ceiling is still there, so its ordering ability survives and the loss is in the level. The decisive number is simpler than the ratio of the gaps: the Ridge’s causal ordering, 0.0984, is more than double the tree’s look-ahead ceiling of 0.0468. A perfect level repair would still leave the tree behind where the Ridge already stands without one.

The persistence contrast agrees. Correlating a block’s realised level with the previous block’s, within one fit and across a refit:

	refit every 84 rows	1 fit, 756-row window
tree	− 0.39	+ 0.65
Ridge	−0.02	+0.12

Same rows, same period, same algorithm; the only difference is whether a refit sat between the two blocks. So does the table of level estimators (dev, 69 folds, standard error 0.120):

estimator	LightGBM	Ridge
the model predicting its own last 84 training rows	+0.118	+ 0.342
the model predicting its whole training set	+0.211	+ 0.415
mean of the training labels	+0.212	+0.366

Nothing clears two standard errors for the tree: its level appears when the model meets the test period and is unreadable beforehand. The same readings correlate for the Ridge. So no estimate can claim the tree’s 0.041 gap, because the level it would have to predict is not readable in advance.

Because the level belongs to the fit, nothing chosen on one span survives to the next. Correlating *vs bh* across the mapping search’s 40 cells, dev to valid:

	Pearson	Spearman
tree	− 0.519	−0.506
Ridge	+ 0.426	+0.376

Choosing a configuration on dev actively hurts the tree on valid. This project’s own historical benchmark for the same number is +0.070, across 37 volatility-overlay candidates.

The consequences show up under the identical shipped mapping:

	dev	valid
tree	+0.089	− 0.069
Ridge	+0.034	+ 0.043

and in the search of §2.8, where configurations clearing dev and valid come to 0 of 210 for the tree against 90 of 210 for the Ridge. The tree clears nothing anywhere in that grid, under any turnover control.

Ship the Ridge. One mechanism explains all three measurements: each of the tree’s fits carries its own output level, a level that belongs to a fit cannot survive being refitted, and so nothing selected on one span carries to the next. The Ridge’s output level barely moves between fits, which is why its configuration space transfers and the tree’s inverts.

That also settles §1.6’s debt. The tree was owed a magnitude encoding, since a rank-only mapping is lossless for the Ridge (*t* on the magnitude = 0.74) and lossy for the tree (*t* = 3.52). The tree is dropped because nothing selected on it transfers, which is a separate finding from the encoding it never got.

A limit on this section. valid is ten blocks, and the tree arm is seven bags at one seed against the Ridge’s closed form. The directions agree across three independent measurements; the individual values are not precise.

► **Leads to.** The model is settled. §2.3 found that removing nothing orders best, so the criterion has to be checked before the length of what to remove can be swept.

2.5 Select on the scored metric, not on the ordering

detrend_sweep.py --grid --axis (Ridge, dev + valid)

span	detrend	block_ic	vs bh	exposure	vol ratio	turnover
dev	none	0.0950	+0.028	1.004	1.211	0.191
dev	roll21	0.0558	+0.060	1.005	1.195	0.298
valid	none	0.089	−0.067	1.006	1.177	0.124
valid	roll8	0.082	+0.384	0.999	1.143	0.344
valid	roll63	0.064	−0.072	0.989	1.160	0.131

The configuration that orders best pays least, on both spans. On valid the two rules have nearly the same ordering, 0.089 against 0.082, the same average exposure, 1.006 against 0.999, and the same realised volatility, 1.177 against 1.143. They differ by 0.45 in adjusted Sharpe. Ordering skill and risk taken are both ruled out by those columns, which leaves when the position moves. block_ic is a rank correlation weighting every day equally, while the metric is carried by a handful of large days, and a rank correlation cannot see the difference.

So block_ic drops to a diagnostic and configurations get chosen on adjusted Sharpe against buy-and-hold. The responsiveness account is inferred from what the other columns exclude rather than measured; a return-weighted IC, or the same comparison split by the size of the day’s move, would test it directly.

► **Leads to.** With the criterion fixed, the length of the removed mean can be swept on the metric.

2.6 The length of the mean that gets removed

mapping_form.py --axes, detrend_sweep.py --axis (dev and valid)

Dev alone does not resolve this. mapping_form.py --axes, pooled on dev, puts mu_window = 21 at the top of every rank-window column, but the paired contrast against 63 is +0.0053 against a detectable of 0.106, twenty times inside the noise floor. Valid can separate the short lengths.

length	5	8	13	21	34	42	63	126	252	none
vs bh	+0.264	+0.384	+0.198	+0.031	−0.025	−0.010	−0.072	−0.082	−0.099	−0.067
turnover	0.399	0.344	0.275	0.217	0.171	—	0.131	—	0.129	0.124

The break sits between 21 and 34. Block-aligned alternatives in the same grid fail on valid in every variant:

detrend	dev	valid
none at all	+0.028	−0.067
mean of the previous 1 complete block	−0.010	−0.074
previous 3 blocks	+0.003	−0.096
previous 6 blocks	−0.008	−0.062
every previous block	+0.017	−0.067
rolling 8	+0.074	+0.384

A trailing mean of 5 to 21 rows. Removing nothing loses money despite ordering best (§2.5); removing a long mean loses money and orders badly; the short end is the only range that does both acceptably. Lengths ≥ 34 are out, so is removing nothing, and so are the block-aligned means. A level decided at refit time and held constant through the block never straddles a boundary, and it is negative on valid in every variant, so the answer is neither “avoid the block boundary” nor “leave the level alone”.

The mechanism stays open. A clean account exists for a model whose level belongs to the fit: a mean of length L reaches into the previous fit on about $L/84$ of days, and consecutive fits’ levels are uncorrelated. §2.4 shows the Ridge is not such a model, so the account does not apply. What is established is the empirical break, not why it sits at 21.

► **Leads to.** The turnover row above is the price of the short end. The next step decides whether to pay it.

2.7 Turnover is the cost, and the metric does not charge it

`detrend_sweep.py --axis (Ridge, valid)`

Turnover is monotone in the length of the mean, from 0.399 at length 5 to 0.124 at no removal. A shorter mean means a faster-moving position, through the same operation. At the short end the rule changes 34 to 40% of notional every day, and unconstrained configurations across the whole grid run from 25 to 50%.

The metric contains no execution. It scores a position series, so a rule trading 40% of notional a day and one trading 5% receive identical treatment. Nothing in the metric constrains turnover, and nothing in dev or valid does either.

Trading 40% of notional a day is not a deployable product whatever the metric says, so the position is required to be smoothed and the parameters are chosen inside that constraint. This is a prior about what kind of rule is worth having. The data does not supply it, and it rules out the unsmoothed family on grounds the data does not support.

► **Leads to.** Given that turnover has to come down, the question is which way of bringing it down destroys the least edge.

2.8 Continuous smoothing gives up the least edge per unit of turnover

`detrend_sweep.py --control (Ridge, dev + valid)`

15 ways of computing the removed mean $\times$ 7 controls $\times$ 2 rank windows, the controls applied to the position after the scale is solved, read as edge retained per unit of turnover spent and averaged over all fifteen means.

control (dev)	<i>vs bh</i>	turnover	edge per unit turnover	
unconstrained	0.059	0.324		0.18
band 0.05	0.044	0.269		0.16
band 0.10	0.030	0.224		0.13
band 0.20	0.000	0.154		0.00
EWMA half-life 2	0.027	0.095		0.28
EWMA half-life 3	0.031	0.070		0.44
EWMA half-life 5	0.035	0.047		0.74

clearing dev and valid, by family	unconstrained	band	EWMA
short mean (3–21)	20 of 22	41 of 66	29 of 66
long mean (34–63)	0 of 6	0 of 18	0 of 18
no mean removed	0 of 2	0 of 6	0 of 6

A no-trade band is dominated at every level. Band 0.20 gives up the entire edge, $+0.000$, to halve turnover, while an EWMA of half-life 5 keeps 59% of it at a seventh of the trading. What each does to the level explains why: a band holds the position unchanged on most days and loses track of the level, while an EWMA moves every day by less and keeps it. The control is an EWMA on the position.

The family separation in the second table is not a turnover effect. The long-mean and no-removal families clear nothing under any control, including no control at all, so §2.6's result holds however turnover is handled.

► **Leads to.** Every parameter now has a value or a range. What remains is choosing among the survivors by a rule fixed in advance.

3 Selection and result

3.1 Selection

`detrend_sweep.py --control` (*dev + valid only, scored by the competition metric with nothing added*)

The smoothing constraint from §2.7 leaves 90 configurations: 15 ways of computing the removed mean $\times$ 3 EWMA half-lives $\times$ 2 rank windows. The pre-registered rule is maximin, maximise the smaller of dev and valid, which requires both spans to stand up and needs no threshold.

removed mean	EWMA half-life	rank window	dev	valid	maximin
ewm3	3	252	+0.036	+0.063	0.036
ewm3	3	504	+0.035	+0.064	0.035
mean8	5	252	+0.034	+0.043	0.034
med8	5	252	+0.034	+0.068	0.034
ewm3	2	252	+0.033	+0.118	0.033
mean5	2	504	+0.033	+0.216	0.033
ewm3	2	504	+0.032	+0.122	0.032
mean5	3	504	+0.032	+0.154	0.032

The top of that ranking is a tie. The first eight rows differ by 0.004 in maximin score, far inside any resolution these spans support, and 29 of the 90 clear both. So three rules are applied and all three are carried forward.

rule	selects	why this rule
A, maximise min(dev, valid)	<code>ewm3</code> $\times$ hl 3 $\times$ rank 252	pre-registered; requires both spans to hold, needs no threshold
B, maximise valid subject to dev $>$ 0	<code>mean5</code> $\times$ hl 2 $\times$ rank 504	uses valid for what valid is for, with dev only as a gate
C, maximise mean(dev, valid)	<code>mean5</code> $\times$ hl 2 $\times$ rank 504	equal weight to both spans; picks the same cell as B
D, best flat-window mean	<code>mean8</code> $\times$ hl 5 $\times$ rank 252	the three ways of computing the mean tie at the top, and a flat window is the plainest form of the operation

B and C agree, so three rules give two configurations. D is added because the choice between an exponentially weighted, a flat-window and a median mean is a tie on the evidence, and a reader reproducing this should see the plainest version. A reported result that depends on which rule was used is worth less than one that does not, so all of them are reported.

► **Leads to.** The rule is now fully specified. What follows is its statement, what the data actually fixed, and the one reading of test.

3.2 The rule

pipeline.py (all three spans)

```
Ridge(alpha=1000) | rolling 756 | refit every 84 | 1132 columns
  preprocessing: ffill -> standardise (degenerate columns pinned)
                  -> clip to +/-10 training SD
    | s(t)
v(t) = s(t) - recent mean of s          remove the slow level, once
    |
p(t) = percentile of v(t) in a trailing window turn it into an order
    |
z(t) = sqrt(3) * (2*p(t) - 1)
    |
tau(t) solved daily on trailing 756 so realised vol ratio = 1.2
    |
w*(t) = clip(1 + tau(t)*z(t), 0, 2)
    |
w(t) = EWMA(w*, a few days)             hold turnover near 10% a day
```

Three settings of the two free lengths and the rank window, one per rule:

rule	removed mean	rank window	position EWMA
A, maximin	exponential, half-life 3	252	half-life 3
B/C, max valid and max mean	flat window of 5	504	half-life 2
D, flat-window mean	flat window of 8	252	half-life 5

Which parameters the data actually fixed.

parameter	value	separable	where
model	Ridge	yes, transfer dev→valid +0.43 against -0.52	§2.4
curve shape	linear	no , 7 forms inside 0.021	§2.1
mean exposure $\bar{w}$	1.00	no , flat over 0.9–1.1	§2.2
volatility budget	1.2	yes	§2.2
scale re-solve cadence	daily	pooled yes (+0.049), per-fold no	§2.2
the removed mean	short, 3–8	family yes (≤ 21 against ≥ 34), within-family no	§2.3–§2.6
rank window	252 or 504	no , both clear	§2.6, §3.1
turnover control	EWMA, hl 2–5	control yes , half-life no	§2.7–§2.8

Half of these are ties, which is why §3.1 carries three configurations. The curve’s shape, the mean exposure, the rank window, the length inside the short family and the half-life that smooths the position are all “any value in this range”. What the data fixed is the model, the volatility budget, the solve cadence, the family the removed mean belongs to, and that the turnover control is an EWMA rather than a band.

3.3 Result

Every row was selected on dev and valid alone, by the correspondingly lettered rule. test was scored afterwards. Buy-and-hold is 0.415/0.968/0.302.

rule	configuration	dev	valid	test
A, maximin	ewm3 × hl3 × rank252	0.451	1.030	0.435
B/C, max valid and max mean	mean5 × hl2 × rank504	0.448	1.183	0.448
D, flat-window mean	mean8 × hl5 × rank252	0.449	1.011	0.399

against buy-and-hold	dev	valid	test	turnover (test)	vol ratio (test)
A	+0.036	+0.063	+0.133	0.106	1.057
B/C	+0.033	+0.216	+0.146	0.151	1.077
D	+0.034	+0.043	+0.096	0.068	1.041

All three clear buy-and-hold on all three spans. Average exposure is 1.00 everywhere, turnover runs 4–15% of notional a day, and both penalty factors come out at exactly 1.000, so realised volatility never approaches the 1.2 kink and the strategy never underperforms the market.

The strongest statement the evidence supports is about the family rather than any one row. Of the 90 smoothed configurations, 29 clear both dev and valid, and all 29 are positive on test: mean +0.129, minimum +0.079, maximum +0.168. The long-mean and no-removal families clear nothing under any turnover control, including none at all.

Three checks made on dev and valid also held when test was read, and none of them entered a decision. The length sweep of \$2.6 keeps its sign in 11 of 12 cells. The cell-to-cell transfer of \$2.4 stays positive for the Ridge, +0.224 from dev and +0.476 from valid, and stays negative for the tree, −0.741 from valid. Under the shipped mapping the tree reaches +0.012 against the Ridge’s +0.096, so the model choice would have been the same had test been available.

The smoothing constraint cost measured score. Under this metric more turnover scores better, and averaged over the grid an unconstrained position beats every smoothed one on both development spans (§2.7). Requiring the position to be smoothed therefore gave up score on dev and valid, on deployability grounds.

3.4 What this is not

The edge is small: +0.036 on dev against a pre-registered threshold of +0.10. It clears buy-and-hold on all three spans for a reason that replicates across a family of configurations, and it is scored on daily equity-index exposure by a metric that charges nothing for trading.

Two mechanisms stay open: why a short mean pays when it orders no better than removing nothing (§2.5), and why the break sits at 21 (§2.6). The account that fitted the LightGBM does not apply to the model that ships.

A The volatility budget is left unspent on purpose

`scale_order.py` (*dev + valid*)

Smoothing comes after the scale is solved, so realised volatility falls from the 1.19 the solver aims at to about 1.06. The penalty never binds and part of the allowance goes unused, which raises whether to reclaim it.

An EWMA is linear, so $\text{EWMA}(\bar{w} + \tau z) = \bar{w} + \tau \text{EWMA}(z)$. Solving the scale on the smoothed signal aims it at what will actually be held. Three arms, the third a control that filters the signal but keeps the old scale.

config	arm	dev	valid	maximin	turnover (dev)	vol ratio
A, maximin	smooth after the solve (shipped)	+0.036	+0.063	0.036	0.087	1.083
	solve on the smoothed signal	+0.019	+0.093	0.019	0.157	1.190
	filter the signal, keep the old scale	+0.036	+0.067	0.036	0.087	1.080
B/C, max valid	smooth after the solve (shipped)	+0.033	+0.216	0.033	0.131	1.090
	solve on the smoothed signal	+0.008	+0.343	0.008	0.222	1.188
	filter the signal, keep the old scale	+0.033	+0.220	0.033	0.131	1.088
D, flat-window	smooth after the solve (shipped)	+0.034	+0.043	0.034	0.057	1.061
	solve on the smoothed signal	+0.019	+0.068	0.019	0.129	1.191
	filter the signal, keep the old scale	+0.034	+0.047	0.034	0.057	1.057

The algebra holds. The third arm matches the first to three decimals on dev in every configuration, and the clip binds on 4–5% of days under the re-solved arm and on none under the shipped one. Re-solving does push the realised ratio from about 1.06–1.09 up to 1.19, and it cuts the maximin score by half or more in all three configurations while roughly doubling turnover. It loses under the same rule everything else was chosen by.

This is §2.2 from the other side. The unpenalised Sharpe saturates near a ratio of 1.3, so a wider swing amplifies whatever is in the signal, including the noise. The gain sits entirely on valid, where the signal is strong, and the loss sits on dev’s 69 blocks. The unspent allowance is not free money and the shipped order stands.

B Reproducing

Every number here is written to `kaggle_hull/probe/` by the script named in its step, and re-reading it needs no refitting: the signal caches are keyed by feature-table row, so a later stage replays without re-running an earlier one.

```
python stage1/Hull_Tactical_feature_engineering.py # the 1132 columns
python tools/split_data.py --commit                # the three slices
python tools/audit_config.py                       # exits with the mismatch count
python stage2/pipeline.py --commit                 # the result
```

`docs/RUNBOOK.md` gives all eighteen commands in order with their costs. `detrend_sweep.py` and `scale_order.py` score dev and valid only; `--reveal-test` opens the third span.